

이 함수에 전역변수를 추가하고 싶은데 어떤걸 넣으면 좋을까

RPG 게임의 볼륨을 키우고 시스템을 더 체계적으로 만들기 위해 전역변수를 도입하려는 생각이시군요! 아주 좋은 접근입니다.

현재 코드에서 매번 사냥할 때마다 스탯 수치를 재계산하거나, 보스 정보가 `main` 함수 안에 갇혀 있는 부분을 전역변수(또는 전역 상도)로 빼면 코드가 훨씬 깔끔해지고 확장하기 좋아집니다.

추가하면 게임이 훨씬 풍성해질 만한 추천 전역변수 아이디어 3가지를 정리해 드립니다.

1. 보스 관련 전역 변수 (가장 추천!)

현재는 첫 번째 보스의 능력치가 `main` 안에 고정되어 있습니다. 보스를 전역변수(또는 배열)로 빼두면, 보스를 잡았을 때 "2단계 보스", "3단계 보스"로 레벨을 올리며 게임을 확장하기 쉬워집니다.

- `int g_boss_level = 1;` : 현재 도전 중인 보스의 단계
- `int g_boss_hp = 800;` : 현재 보스의 체력
- `int g_boss_power = 1300;` : 현재 보스의 전투력

💡 **적용 예시:** 보스 레이드에서 승리하면 `g_boss_level++`; 을 하고, 보스의 hp와 power를 각각 1.5배씩 증가시키는 형태로 업그레이드할 수 있습니다.

2. 게임 내 재화 (골드/돈) 시스템

RPG 게임의 핵심인 '상점'이나 '강화'를 추가하기 위한 전역변수입니다. 사냥할 때마다 골드를 얻고, 보스를 잡으면 큰 돈을 주도록 설계할 수 있습니다.

- `int g_player_gold = 0;` : 플레이어가 보유한 총 골드

💡 **적용 예시:** 사냥 성공 시 `int gold_gain = 50; g_player_gold += gold_gain;` 을 해주고, 나중 에 메뉴에 "4. 상점 입장 (골드로 스탯 구매)" 항목을 새로 만들 때 기반이 됩니다.

+ Gemini에게 물어보기

Flash v



Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. [개인 정보 보호 및 Gemini](#)

3. 캐릭터 직업별 '보너스 계수' 상수 (코드 깔끔화)

현재 전사, 마법사, 성직자의 스탯 계산 공식(`1.5` , `1.3` , `2.2` 등)이 캐릭터 생성할 때와 레벨업할 때 코드 곳곳에 중복해서 적혀 있습니다. 이걸 전역 상수로 고정해 두면 관리하기가 엄청나게 편해집니다.

- `const double WARRIOR_HP_RATIO = 1.5;`
- `const double WARRIOR_AD_RATIO = 1.3;`
- `const double MAGE_AP_RATIO = 2.2;`

전투력 계산부분을 함수로 만들고 싶은데 어떻게 만들어야할까

전투력 계산 부분을 함수로 만들면 코드의 중복을 획득 줄일 수 있고, 나중에 새로운 직업이 추가되거나 밸런스를 패치할 때도 그 함수 한 곳만 고치면 되기 때문에 아주 훌륭한 선택입니다!

이 함수를 만들 때는 "직업 번호", "체력", "공격력", "마력"을 매개변수(입력값)로 넘겨주고, 계산된 "총 전투력 (double)"을 반환(return)받도록 설계하는 것이 가장 좋습니다.

구현하는 방법과 메인 코드에 적용하는 법을 알기 쉽게 정리해 드릴게요.

1. 전투력 계산 함수 정의

`main` 함수 위쪽에 다음과 같이 함수를 새로 만듭니다. 성직자의 신앙심(`faith`) 같은 중간 계산도 이 안에서 처리하여 `main` 코드를 깔끔하게 만들 수 있습니다.

C

```
// 전투력을 계산하여 반환하는 함수
double calculate_power(int job, int hp, double ad, double ap) {
    double power = 0.0;
    double faith = 0.0;

    switch (job) {
        case 1: // 전사
            power = ad * 1.3 + hp * 1.5;
            break;
        case 2: // 마법사
            power = ap * 2.2 + hp * 0.9;
            break;
        case 3: // 성직자
            faith = 1.4 * ad + 1.5 * ap;
            power = faith + hp;
            break;
        default:
            power = 0.0;
            break;
    }
}
```

+ Gemini에게 물어보기

Flash



2. 사냥터 기능 함수 (강력 추천)

현재 `case 2`: 내부에는 경험치 획득, 레벨업 판단, 직업별 스탯 증가 등 엄청나게 복잡한 로직이 들어가 있습니다. 이 사냥터 기능 전체를 하나의 함수로 독립시킬 수 있습니다.

사냥을 하면 레벨과 경험치, 체력, 공격력 등이 바뀌어야 하므로 이 값들을 함수 안에서 수정할 수 있도록 연동해야 합니다. 복잡한 포인터(`*`) 개념을 쓰기 어렵다면, 사냥 횟수를 넘겨받아 '획득할 총 경험치'를 반환하는 형태나 스탯을 전역변수로 빼고 함수로 제어하는 형태가 좋습니다.

만약 모든 변수를 `main` 에 두고 함수로만 분리하고 싶다면 포인터(주소 전달)를 활용하게 됩니다.

- 함수 이름 추천: `void enter_hunting_ground(int job, int *level, int *exp, int *hp, double *ad, double *ap, double *power)`

3. 보스 레이드 전투 임무

보스 레이드 부분도 따로 떼어내기 아주 좋은 단위입니다. 플레이어의 전투력과 보스의 정보를 던져주고 승리 여부만 판별해 오면 됩니다.

- 함수 이름 추천: `void start_boss_raid(double player_power, int boss_hp, int boss_power)`

C

```
void start_boss_raid(double player_power, int boss_hp, int boss_power) {
    int try_boss;
    printf("보스레이드를 시작합니다!\n");
    printf("=====\n");
    printf("보스의 체력: %d | 전투력: %d\n", boss_hp, boss_power);
    printf("현재 내 전투력: %.2lf\n", player_power);
    printf("도전하시겠습니까? (예:1, 아니오:2): ");
    scanf("%d", &try_boss);

    if(try_boss == 1) {
        printf("전투를 시작합니다...\n");
        if(player_power >= boss_power) {
            printf("🎉 승리! 보스를 물리쳤습니다!\n");
        } else {
            printf("🎭 패배... 힘이 부족합니다.\n");
        }
    } else {
        printf("게임을 중지하고 메뉴로 돌아갑니다.\n");
    }
}
```

💡 요약하자면

메뉴 출시는 아주 간단하게 바꿀 수 있으니 지금 바로 적용해 보시는 것을 추천합니다!

사냥터나 보스 레이드 같은 대형 기능을 함수로 뉘 때는 "변수들을 포인터(*)로 주고받을 것인가", 아니면 아까 이야기 나눈 "전역 변수로 만들어 함수들이 공유하게 만들 것인가"를 먼저 결정해야 코딩이 쉬워집니다. RPG 게임 특성상 스탯 변수가 워낙 많기 때문에, 핵심 스탯을 전역 변수로 빼고 함수화하는 것이 형찬 님이 코드를 짜기에 훨씬 직관적이고 편할 거예요.

이 세 가지 중에서 어떤 기능을 먼저 함수로 옮겨보고 싶으신가요? 포인터 방식과 전역변수 방식 중 원하시는 스타일을 알려주시면 맞춤형 코드로 가이드해 드릴게요!

+ Gemini에게 물어보기

Flash ▾



위의 보스 레벨 전역변수를 이용해서 총 5단계의 보스 레이드를 만들려면 각 단계의 보스 도전 양식을 일일이 다 만들어야해?

아니요! 일일이 다 만들 필요가 전혀 없습니다. 바로 그 번거로운 작업을 안 하려고 우리가 전역변수와 반복문/수식을 쓰는 것이니까요!

5단계까지 보스 레이드를 만들기 위해 case 3: 안에 보스 1, 보스 2, 보스 3... 코드를 복사.붙여넣기 할 필요 없이, 전역변수의 값만 승리할 때마다 자동으로 가짜 레벨업(스텝 업)을 시켜주면 단 하나의 코드로 5단계(그 이상도 가능)까지 무한히 재사용할 수 있습니다.

어떻게 코드를 한 줄도 늘리지 않고 5단계 보스를 만드는지 원리와 완성된 코드를 보여드릴게요.

1. 핵심 원리: 승리 시 보스 스텝 스케일업(Scale-up)

기존에는 보스 능력치가 고정되어 있었지만, 전역변수를 활용하면 다음과 같이 설계할 수 있습니다.

- 보스 승리 시: 1. 보스 레벨을 1 올린다 (`g_boss_level++;`)
2. 다음 단계 보스의 체력과 전투력을 일정 비율(예: 1.5배)로 성장시킨다.
- 최종 단계(5단계) 승리 시: "울클리어! 게임 승리!" 메시지를 띄우고 종료하거나 초기화한다.

2. 전역변수를 도입한 5단계 보스 레이드 통합 코드

형찬 님이 이전에 만든 코드에 보스 전역변수를 상단에 추가하고, case 3: 의 보스전 로직을 시스템화한 전체 코드입니다.

이 방식의 장점 🌟

- 보스전 양식을 일일이 복사할 필요 없이 코드 양이 그대로 유지됩니다.
- `g_boss_hp = (int)(g_boss_hp * 1.5);` 부분의 수치(1.5)만 바꾸면 보스의 난이도를 아주 쉽게 밸런스 패치할 수 있습니다.
- `if (g_boss_level > 5)` 에서 5 를 10 으로만 바꾸면 순식간에 10단계 보스 레이드 게임으로 확장도 가능해집니다!